

Deep Learning

Lecture 2: PyTorch programming

Chris G. Willcocks

Durham University



Learning objectives

By the end of this lecture you should be able to:

- Understand the foundations of PyTorch
- Represent data and batches as tensors.
- Relate a model's parameters to its predictions and loss.
- Explain how gradients are used to train a model.
- Evaluate predictions on examples excluded from training.



Definition: tensor

In numerical programming, a tensor is an array of numerical values. A scalar has no axes, a vector one and a matrix two.

The **shape** gives the size of each axis. An image batch has axes for examples, channels, height and width.

A batch groups examples so that we can apply the same model to them together.

Example: MNIST

For a batch of B greyscale images:

$$\mathbf{X} \in \mathbb{R}^{B \times 1 \times 28 \times 28}.$$

Flattening each image gives a matrix:

$$\mathbf{X} \in \mathbb{R}^{B \times 784}.$$

Each row contains one image's 784 pixel intensities. Flattening preserves their values and order.

Reading: D2L §2.1 Data manipulation and §2.3 Linear algebra.

Definition: model and loss

A model f_{θ} maps an input to a prediction. Its parameters θ are learned from data.

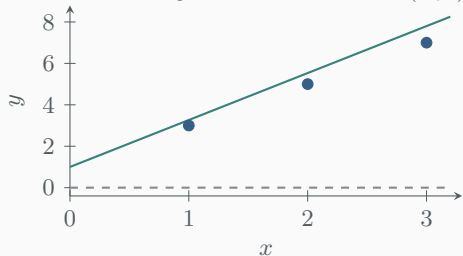
For n training pairs (x_i, y_i) , mean squared error is

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2.$$

Training adjusts the parameters to reduce this loss on the training examples.

Reading: D2L §3.1 Linear regression.

Linear model: $\hat{y} = wx + b$, with $\theta = (w, b)^T$



• data

- - - initial model

— after one update

Forward, backward, update

1. **Forward:** compute predictions and a scalar loss.
2. **Backpropagation:** apply the chain rule backwards to find how the loss changes with each parameter.
3. **Update:** take a small step in the direction of the negative gradient.

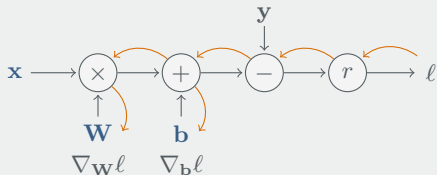
For a batch with mean loss $\mathcal{L}_{\mathcal{B}}$, gradient descent gives

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{B}}(\theta).$$

The learning rate η controls the step size.

Example: a linear layer

For input $\mathbf{x}$ and target $\mathbf{y}$, the layer predicts $\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}$.



Subtract the target, then square and sum the errors at r to obtain $\ell = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$. Orange arrows give loss gradients for weights $\mathbf{W}$ and bias $\mathbf{b}$.

Reading: D2L §2.5 Automatic differentiation and §5.3 Backpropagation.



Model: digit classifier

The input is a vector $\mathbf{x} \in \mathbb{R}^{784}$ of pixel intensities. A hidden layer computes

$$\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}), \quad \mathbf{h} \in \mathbb{R}^{64}.$$

The output gives ten class scores, or **logits**:

$$\mathbf{z} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}, \quad \mathbf{z} \in \mathbb{R}^{10}.$$

ReLU applies $\max(0, a)$ to each entry. Removing it would leave a single affine model.

Loss: cross entropy

Softmax converts scores to probabilities:

$$p_k = \frac{\exp(z_k)}{\sum_{j=0}^9 \exp(z_j)}, \quad k \in \{0, \dots, 9\}.$$

For the correct digit $y \in \{0, \dots, 9\}$,

$$\ell(\mathbf{z}, y) = -\log p_y.$$

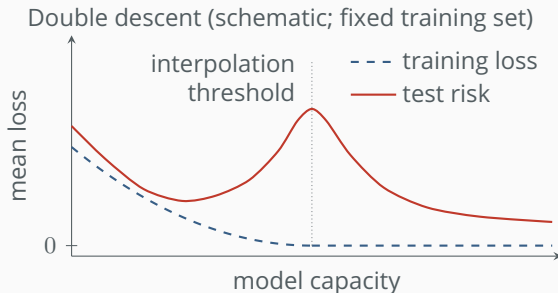
A correct-class probability of 0.9 gives loss 0.11; 0.1 gives 2.30. Accuracy uses the highest-scoring class.

Reading: D2L §4.1 Softmax regression and §5.1 Multilayer perceptrons.

Evaluation on unseen examples

Risk is the expected loss on new examples from the data distribution.

- **Training:** fit the parameters.
- **Validation:** choose the model and training settings.
- **Test:** estimate risk after those choices are fixed.



Test risk can fall again after the model first fits all training examples. The shape depends on noise, regularisation and optimisation.

Reading: D2L §3.6; Belkin et al. (2019); Nakkiran et al. (2020).

Predicting an image

The input is a greyscale image; the target is the original colour image:

$$f_{\theta} : \mathbb{R}^{1 \times 32 \times 32} \rightarrow \mathbb{R}^{3 \times 32 \times 32}.$$

We will train an MLP encoder-decoder using mean squared error over the RGB values.

Several colours can give the same grey value. The training images provide information about likely colours, but the original colour cannot always be recovered.

Reading: D2L §5.1 Multilayer perceptrons.



Greyscale inputs, model predictions and RGB targets, from top to bottom.